JAVA 2

Module TMP

Dates et heures

Au programme

1. Les classes de `java.time`
2. Mesurer un écart
3. Une date ne change pas
4. L'affichage formaté
5. Date ou instant ?
6. Arithmétique du calendrier
7. `Period` ou `Duration`
8. Une date et son texte

1. Les classes de `java.time`



Une date, c'est plus compliqué qu'il n'y paraît

Combien de jours dans un mois ? Ça dépend…

Combien de jours dans une année ? Ça dépend…

Le 31 février existe-t-il ? Non, mais rien n'empêche de l'écrire

Faire soi-même ce calcul, c'est se tromper

Les trois classes du quotidien

La classe `LocalDate` - une date, sans heure - `2027-01-19`

La classe `LocalTime` - une heure, sans date - `18:00`

La classe `LocalDateTime` - les deux - `2027-01-19T18:00`

« Local », donc sans fuseau horaire

Seul `now()` consulte celui de la machine, pour savoir quelle date il est

Laquelle choisir ?

Ce que vous représentez	La classe
une date de naissance, une échéance	`LocalDate`
une heure d'ouverture	`LocalTime`
un rendez-vous, un horodatage	`LocalDateTime`
un écart en années, mois et jours	`Period`
un écart en heures, minutes et secondes	`Duration`

Tout est dans le JDK

Le package `java.time` fait partie de la bibliothèque standard

Rien à installer, rien à déclarer dans le `pom.xml`

Le détail : mémento technique, **`TEC-PLA.1`**

Fabriquer une date

Aucun constructeur : on utilise des méthodes statiques

La méthode `parse` attend le format ISO, `AAAA-MM-JJ`

```
LocalDate aujourdhui = LocalDate.now();
LocalDate naissance  = LocalDate.of(1990, 5, 17);
LocalDate lue        = LocalDate.parse("1990-05-17");
```

Lire un morceau de date

Les noms sont prévisibles, la complétion suffit

La méthode `getDayOfWeek` retourne une valeur de type `DayOfWeek`

Une énumération, pas une classe - les énumérations seront vues au module **ENU**

```
int annee           = naissance.getYear();
int jourDuMois      = naissance.getDayOfMonth();
DayOfWeek jour      = naissance.getDayOfWeek();
boolean bissextile = naissance.isLeapYear();
int joursDuMois     = naissance.lengthOfMonth();
```

Comparer deux dates

Pas de < ni de > : ce sont des objets

```
naissance.isBefore(aujourdhui);   // true
naissance.isAfter(aujourdhui);    // false
naissance.isEqual(aujourdhui);    // false
```

Quelques exemples avec `LocalDate`

Remarquez les méthodes en *italique*… Méthodes statiques.

Objectif	Code
Récupérer la date du jour	`LocalDate aujourdhui = LocalDate.now();`
Récupérer une date donnée (15/06/2008)	`LocalDate.parse("2008-06-15");` `LocalDate.of(2008, 6, 15);`
La date de demain	`LocalDate demain = LocalDate.now().plusDays(1);`
Aujourd'hui moins 1 mois	`LocalDate memeJourMoisPrecedent = LocalDate.now().minus(1, ChronoUnit.MONTHS);`
Récupérer le jour de la semaine ou le jour du mois	`DayOfWeek jeudi = LocalDate.parse("2010-07-15").getDayOfWeek();` `int deux = LocalDate.parse("2012-09-02").getDayOfMonth();`
Année bissextile ?	`boolean estBissextile = LocalDate.now().isLeapYear();`
Nombre de jours du mois Nombre de jours de l'année	`int joursDansMois = aujourdhui.lengthOfMonth();` `int joursDansAnnee = aujourdhui.lengthOfYear();`
Nombre de jours entre dates	`long ageEnJours = ChronoUnit.DAYS.between(naissance, aujourdhui);`

L'heure seule

Mêmes conventions que `LocalDate`

Des constantes : `LocalTime.NOON`, `LocalTime.MIDNIGHT`

```
LocalTime debut = LocalTime.of(18, 0);
LocalTime fin   = LocalTime.parse("21:35");
int heure       = debut.getHour();
```

Quelques exemples avec `LocalTime`

`LocalTime` représente une heure, sans notion de date.

Objectif	Code
Récupérer l'heure courante	`LocalTime maintenant = LocalTime.now();`
Récupérer une heure donnée	`LocalTime.parse("18:47");` `LocalTime.of(18, 47);`
Une heure plus tôt	`LocalTime uneHeurePlusTot = maintenant.minus(1, ChronoUnit.HOURS);`
Récupérer l'heure (sans les minutes)	`int heure = maintenant.getHour();`
Savoir si une heure est avant une autre	`boolean estAvant = uneHeurePlusTot.isBefore(maintenant);`
Quelques constantes	`LocalTime.MAX, LocalTime.MIN, LocalTime.MIDNIGHT, LocalTime.NOON`

Les deux à la fois

La classe `LocalDateTime` sait retourner chacune de ses moitiés

```
LocalDateTime rdv = LocalDateTime.of(2027, 1, 19, 18, 0);
LocalDateTime now = LocalDateTime.now();

rdv.toLocalDate();   // 2027-01-19
rdv.toLocalTime();   // 18:00
```

Démonstration



Démonstration

Explorer une classe du JDK sans quitter l'éditeur

Déroulé : Demo_TMP

2. Mesurer un écart



`ChronoUnit`, l'unité de mesure

L'unité contient la méthode, pas l'inverse

On écrit `ChronoUnit.DAYS.between`, et pas `ChronoUnit.between.DAYS`…

```java
import java.time.temporal.ChronoUnit;

long jours = ChronoUnit.DAYS.between(naissance, aujourdhui);
long mois  = ChronoUnit.MONTHS.between(naissance, aujourdhui);
```

Deux pièges de comptage

La méthode `between` compte des unités **entières**

Par conséquent, du 1er au 30 janvier : `MONTHS.between` retourne **0**

L'ordre des arguments décide du signe : on peut récupérer un nombre négatif…

Le résultat est un `long`, pas un `int` (c'est logique si on compte en secondes…)

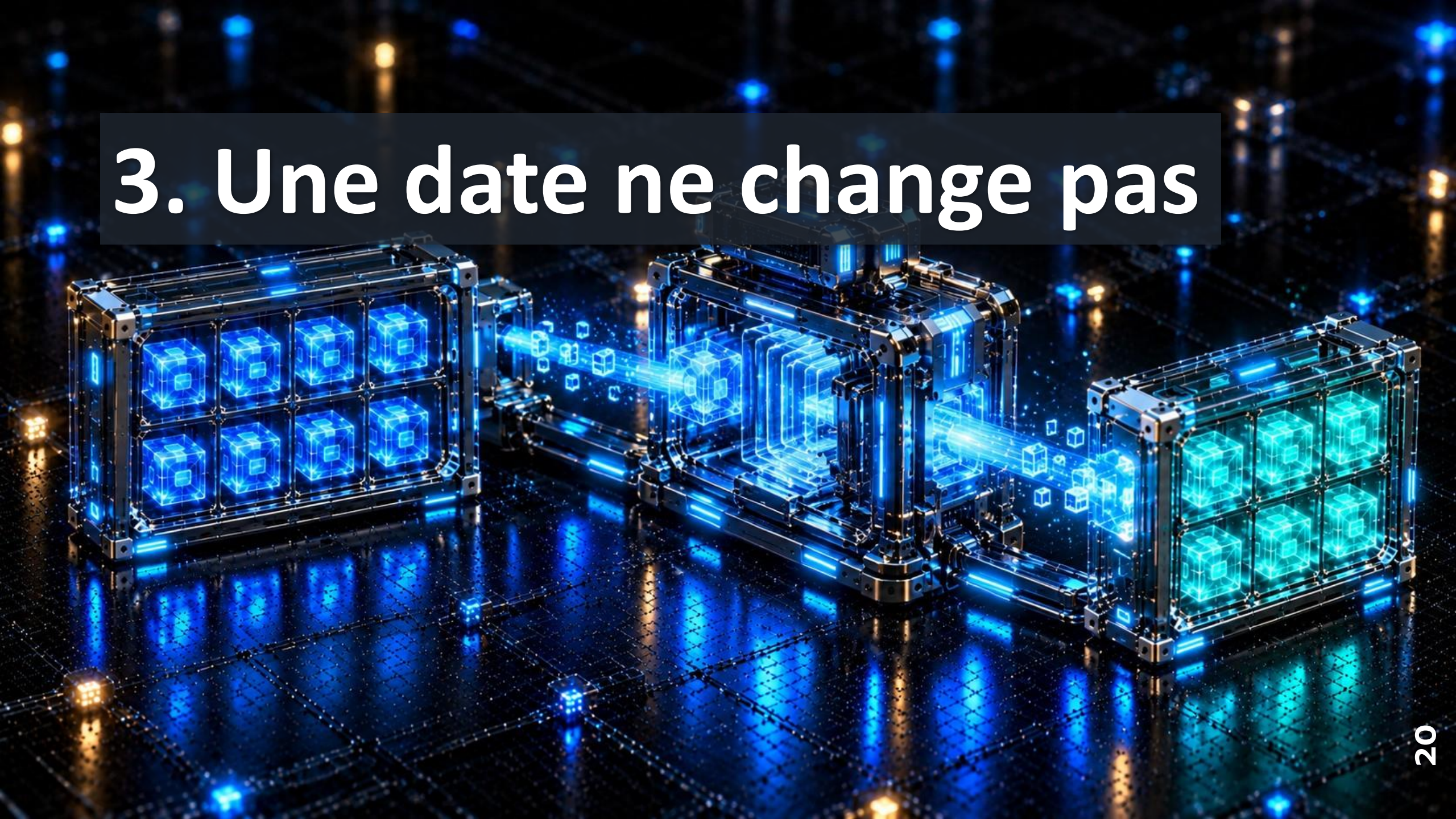
Exercices



TMP.1 Mesurer un écart entre deux dates

fascicule TMP

3. Une date ne change pas



Décaler une date

Des noms limpides, un comportement à vérifier

Qu'affiche la dernière ligne ?

Essayez d'abord de deviner le résultat…

Ensuite, exécutez le code pour vérifier votre intuition

```
LocalDate rendezVous = LocalDate.of(2027, 1, 19);

rendezVous.plusWeeks(1);
System.out.println(rendezVous);   // ?
```

Exercices



TMP.2 Les dates sont immuables

fascicule TMP

Exécutez d'abord le `Programme` fourni, et lisez sa sortie

Retour sur « Décaler une date »

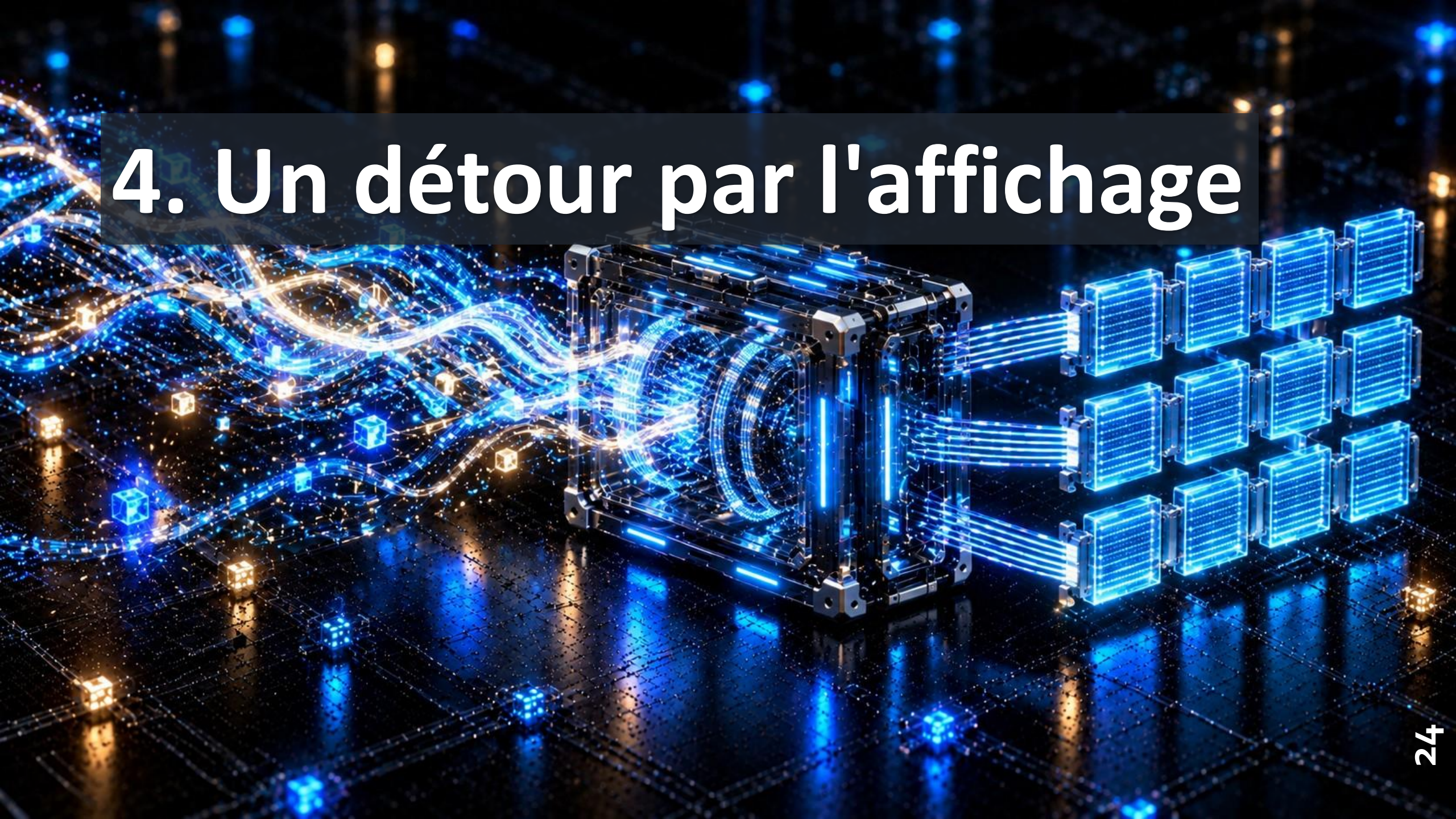
La méthode `plusWeeks` ne modifie rien : elle **retourne** une nouvelle date

Si cette nouvelle date n'est pas affectée à une variable ou utilisée, elle est perdue

Même mécanique que `String.toUpperCase()`

Ce n'est pas une gêne, c'est une garantie - **CC-MET.7**

4. Un détour par l'affichage



`println` et `print`

La méthode `print` sert dès qu'on construit une ligne morceau par morceau

```
System.out.println("Bonjour");   // affiche et passe a la ligne
System.out.print("Bonjour");     // affiche, sans passer a la ligne

System.out.print("A");
System.out.print("B");           // => AB
```

`printf` : un gabarit et des valeurs

Le premier argument est un **gabarit**

Chaque trou est rempli dans l'ordre

`%n` : le retour à la ligne

```
System.out.printf("%s a %d ans%n", "Max", 21);
// Max a 21 ans
```

Les gabarits utiles



Gabarit	Résultat	Effet
`%d`	`1234567`	un entier
`%,d`	`1,234,567`	séparateur de milliers
`%s`	`Max`	du texte
`%f`	`3.141590`	six décimales par défaut
`%.2f`	`3.14`	deux décimales
`%8.2f`	`  3.14`	deux décimales, sur 8 caractères

La locale décide du séparateur

Sans locale, le résultat dépend de la machine

Un test qui compare des chaînes échouera sur un espace invisible

```
System.out.printf("%,d%n", 1157479200);
// machine francaise : 1 157 479 200  (espace INSECABLE etroite)
// machine anglaise  : 1,157,479,200

System.out.printf(Locale.US, "%,d%n", 1157479200);
// partout           : 1,157,479,200
```

`String.format`, la version testable

Mêmes gabarits, deux usages

La méthode `printf` affiche - on ne peut pas la tester

La méthode `String.format` retourne - on peut la tester

Fin du détour : on revient à la mesure du temps

```
System.out.printf(Locale.US, "%,d", n);          // affiche
String s = String.format(Locale.US, "%,d", n);   // retourne
```

5. Date ou instant ?



Ce que `LocalDate` ne contient pas

Un `LocalDate`, c'est une case du calendrier

Pas d'heure, pas de minute, pas de seconde

Que donne la dernière ligne ?

```
ChronoUnit.DAYS.between(d1, d2);      // fonctionne
ChronoUnit.SECONDS.between(d1, d2);   // ?
```

Exercices



TMP.3 Compter des secondes : date ou instant ?

fascicule TMP

Décommentez la ligne signalée et **lisez le message**

Retour sur « Ce que `LocalDate` ne contient pas »

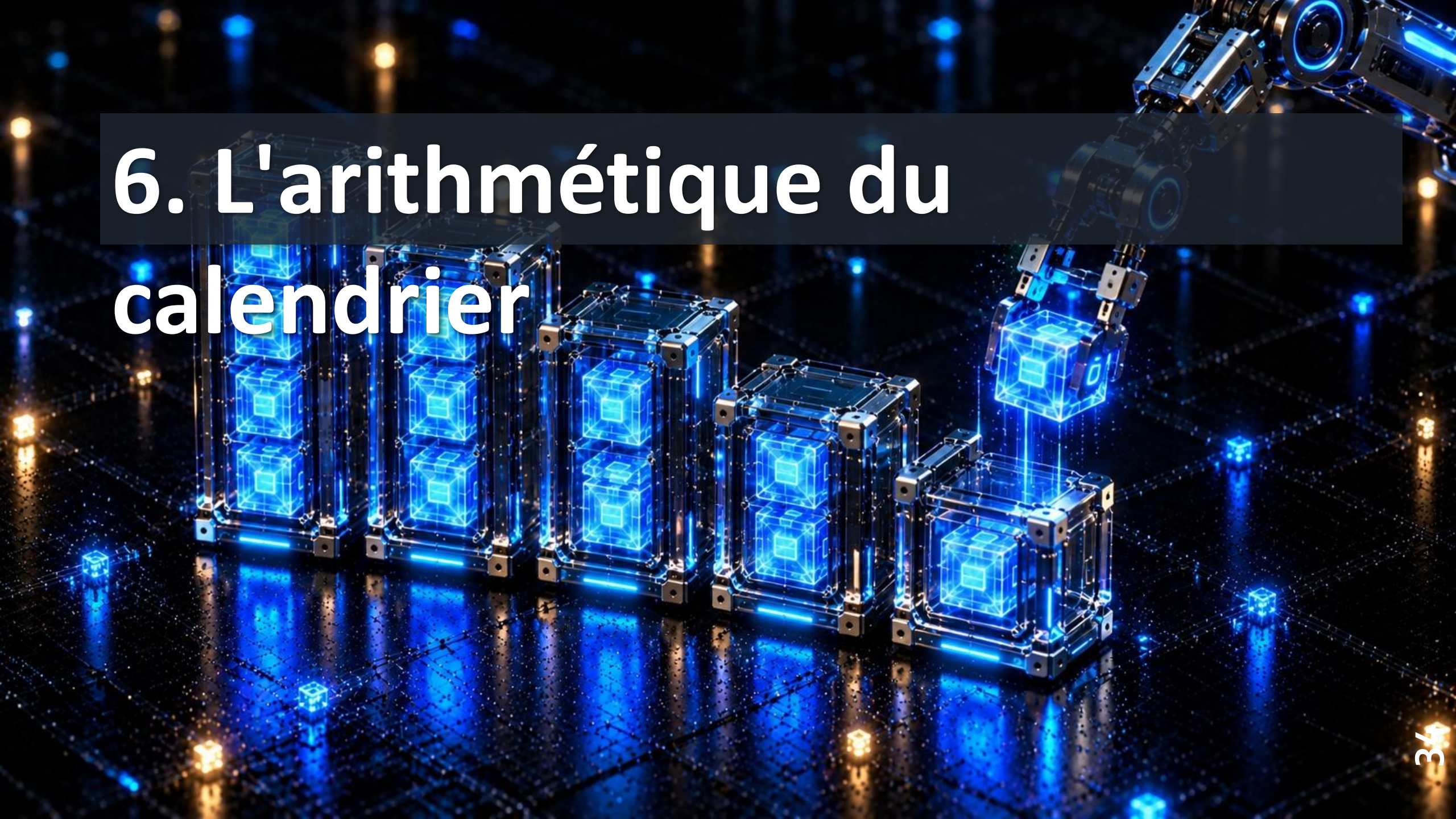
Le JDK refuse plutôt que d'inventer : `Unsupported unit: Seconds`

La méthode `atStartOfDay()` fixe minuit - une convention, pas une vérité

Votre résultat est donc juste à quelques heures près

Un `int` s'arrête à 2 147 483 647, soit environ 68 ans de secondes

6. L'arithmétique du calendrier



Ajouter des mois

Les deux dernières lignes donnent-elles la même date ?

Votez avant d'exécuter

```java
LocalDate d = LocalDate.of(2027, 1, 31);

d.plusDays(30);
d.plusMonths(1);
d.plusMonths(1).plusMonths(1);   // ?
d.plusMonths(2);                 // ?
```

Exercices



TMP.4 L'arithmétique des mois

fascicule TMP

Retour sur « Ajouter des mois »

Le `31/01/2027` plus un mois donne `28/02/2027`

Le 31 février n'existe pas : Java rabote au dernier jour disponible

Une fois raboté, le 31 est **perdu**

Ajouter deux fois un mois n'est donc pas ajouter deux mois

7. Period ou Duration



`Period`, les unités du calendrier

Années, mois, jours - des unités de longueur variable

Pour des `LocalDate`

```
Period p = Period.between(naissance, aujourdhui);
p.getYears();    // 36
p.getMonths();   // 8
p.getDays();     // 2
```

`Duration`, les unités de l'horloge

Heures, minutes, secondes - toujours identiques

Pour des `LocalDateTime`

Que vaut alors `toHours() + "h" + toMinutes()` ?

```
Duration d = Duration.between(debut, fin);
d.toHours();     // 3
d.toMinutes();   // 215
```

Exercices



TMP.5 Period ou Duration

fascicule TMP

Retour sur « `Duration`, les unités de l'horloge »

La méthode `toMinutes()` retourne le **total**, 215

La méthode `toMinutesPart()` retourne le **reste**, 35

La complétion montre les noms ; la Javadoc dit ce qu'ils font

8. Une date et son texte



De la date vers le texte

dd jour, MM mois, yyyy année

Attention : mm est la minute, MM le mois

```
DateTimeFormatter fr = DateTimeFormatter.ofPattern("dd/MM/yyyy");
String texte = date.format(fr);    // "19/01/2027"
```

Du texte vers la date

Le même formateur sert dans les deux sens

Un texte mal formé lève une exception - les exceptions : module **EXC**

```
DateTimeFormatter fr = DateTimeFormatter.ofPattern("dd/MM/yyyy");
LocalDate d = LocalDate.parse("19/01/2027", fr);
LocalDate e = LocalDate.parse("19-01-2027", fr);   // exception
```

Exercices



TMP.6 Formater et analyser une date

fascicule TMP

Analysez `"31/02/2027"` et regardez ce que vous obtenez

Retour - l'indulgence silencieuse

L'appel `parse("31/02/2027")` retourne `2027-02-28`, sans rien signaler

Mode par défaut : indulgent, il rabote

La méthode `withResolverStyle(STRICT)` refuse - et réclame `uuuu` au lieu de `yyyy`

Mieux vaut une exception qu'une valeur arrangée - **CC-ERR.1**

Ce qu'il faut retenir

Trois classes : `LocalDate`, `LocalTime`, `LocalDateTime`

Une date est **immuable** : les méthodes retournent, elles ne modifient pas

Une date n'est pas un instant

Utilisez `Period` pour des dates, `Duration` pour des heures

La méthode `String.format` se teste, `printf` non

Précisez la locale, et choisissez le mode strict